A Long-Horizon Policy Composer for MABEL: the Choreography Timeline and a Live-Validated Whole-Robot Motion Library

MABEL project

September 15, 2026

Abstract

The Simulation Control Center's timeline is a non-linear-editor (NLE) style *long-horizon policy composer*: two labeled lanes — NAVIGATION (base motion) and MANIPULATION (arms, hands, torso, lift) — on which *action blocks* are arranged, bladed, trimmed, snapped, and duplicated like clips in a video editor, then executed on the robot as one choreography. Actions come from a typed registry with three families: (i) a library of hand-authored, live-validated whole-robot animation clips (emotes, dances) and scripted loco-manipulation task policies; (ii) parametric navigation primitives calibrated against the simulated plant, plus a REST-backed closed-loop *move-to-goal*; and (iii) static poses that double as interpolation keyframes. Aspirational learned-policy slots appear greyed out and are structurally non-executable. Every clip is expressed as a sum of eased envelopes with an enforced $|\dot{q}| < 2.5\,\text{rad/s}$ budget at 20 Hz sampling, starts and ends at the robot's neutral pose, and was iterated against the *live* simulator with visual inspection until it reads as the intended gesture. We quantify the stack: open-loop nav primitives land within 2–10% (e.g. 0.491 m for a commanded 0.5 m translation, 88.5° for 90°), a full 360° dance pirouette returns within 2.4°, every animation returns to neutral within 0.05 rad on all 14 arm joints, and streaming `navJoystick` zeros during manipulation pins base drift from arm reaction forces from 0.221 m to 0.002 m. A location-aware validator walks the timeline, tracks the robot's implied position, flags actions scheduled at the wrong place, and (by default) auto-inserts sized move-to-goal blocks. 95/95 unit tests pass; every figure regenerates from `experiments/exp_choreography.py`.

Keywords: robot choreography, non-linear editing, motion library, procedural animation, easing, teleoperation, long-horizon composition, digital twin.

1 Overview

Long-horizon robot behavior is composed, not monolithic: "go to the kitchen, pick up the cup, come back, celebrate" is a *timeline* of heterogeneous actions — learned policies, scripted skills, navigation legs, expressive emotes — each of which is individually simple. The composer treats this timeline exactly like a video edit. The mental model is deliberately that of a cinema NLE (the project's Data Curation studio): a time ruler, a playhead, lanes, blocks with in/out points, blade and trim tools, deterministic snapping, and persistence. The differences from a video editor are the payloads (action commands, not media) and the run semantics (a 20 Hz command stream to a real control stack, §5).

Three code artifacts implement it, all under `simulation_control_center/web/`:

- `js/motions.js` — pure data + math (node-testable, no DOM): the action registry (§3), the easing toolkit (§2), the `ChoreoTrack` block model (§4), and the location-gap validator (§6).
- `js/timeline.js` — the editor UI and the 20 Hz executor; also the v1 setpoint-keyframe feature, preserved unchanged on the manipulation lane.
- `tests/js/motions.test.mjs` — the machine-checkable part of the quality gate (§8.5).

2 Motion representation

2.1 Clips as sums of eased envelopes

A clip is a pure function $q(t) : [0, d] \to \mathbb{R}^J$ over canonical joint targets, built from four primitives (Fig. 1). With the cosine ease $E(u) = \frac{1}{2} - \frac{1}{2}\cos(\pi\,\mathrm{clamp}_{[0,1]}(u))$:

$$\mathrm{seg}(t; t_0, t_1) = E\left(\frac{t - t_0}{t_1 - t_0}\right) \qquad \text{(eased ramp } 0 \to 1\text{)} \qquad (1)$$

$$\mathrm{win}(t; t_0, t_1, r) = \min\big(\mathrm{seg}(t; t_0, t_0{+}r),\ 1 - \mathrm{seg}(t; t_1{-}r, t_1)\big) \qquad \text{(rise–hold–fall envelope)} \qquad (2)$$

$$\mathrm{osc}(t; f, t_0, t_1, r) = \mathrm{win}(t; t_0, t_1, r)\,\sin\big(2\pi f(t - t_0)\big) \qquad \text{(windowed oscillation)} \qquad (3)$$

$$\mathrm{breath}(t; t_0, t_1) = \mathrm{osc}(t;\ f{\approx}0.28, t_0, t_1, r{=}0.8) \qquad \text{(idle secondary motion)} \qquad (4)$$

Each joint target is a linear combination of these, $q_j(t) = \sum_k A_{jk}\,\phi_k(t)$, so $q(0) = q(d) = 0$ (the neutral pose) holds *by construction* whenever every envelope opens after $t{=}0$ and closes before $t{=}d$ — the property the tests assert exactly (§8.5). Naturalness comes from the classical animation ingredients [1]: anticipation (a wind-up window preceding the main action, e.g. the fist-bump coil), follow-through (envelopes on secondary joints outlast the primary), asymmetry (the off-arm relaxes down during a wave rather than mirroring), and ever-present low-amplitude breathing during holds.

2.2 The velocity budget

The stack-wide safety budget is $|\dot{q}| < 2.5$ rad/s, checked at the executor's own 20 Hz sampling. An eased ramp of amplitude A over rise time r has peak rate $\dot{q}_{\max} = A\pi/2r$, so every envelope obeys $r \geq A\pi/5$ s ($A{=}2.0$ rad $\Rightarrow$ $r \geq 1.26$ s); whole-hand grips are budgeted at their worst-case finger travel of 1.9 rad per grip unit. The test suite walks every sampled trajectory and asserts the bound per tick; Fig. 2 shows the shipped library's headroom (maximum 2.48 rad/s, the `inspect` raise).

2.3 Measured plant conventions

The clip amplitudes are meaningless without the plant's sign conventions, which were established *empirically* against the live simulator (single-joint probes with full-pose commands, state readback, and rendered verification; 2026-07-02 sessions). Table 1 is the reference.

3 The action registry

Every action is one entry of `MOTION_CLIPS`: {`title, emoji, dur`|`durFor(params), lane, category, status, hold?, params?, location?, kind?, joints(t,p)?, nav(t,p)?`} with `lane` ∈ {`manip`, `nav`, `both`}, `category` ∈ {`animation`, `policy`, `nav`, `pose`} (the three menu sections merge animation+policy), and `status` ∈ {`available`, `coming_soon`}. The registry rule is bright-line: *anything addable executes today; anything else is visibly coming-soon* — coming-soon entries carry no executable functions at all, and the model refuses to instantiate blocks for them.

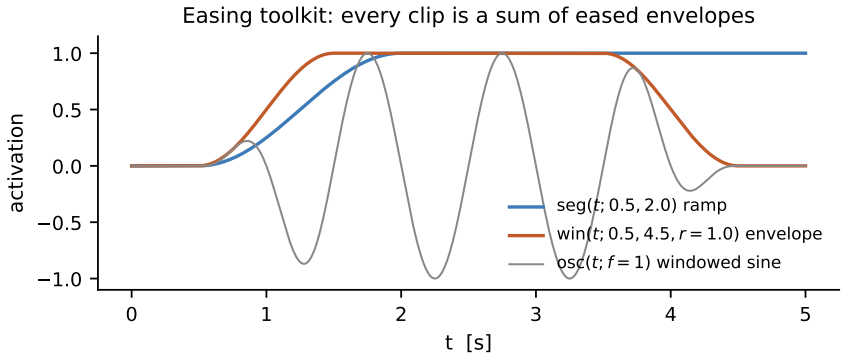


Figure 1: The easing toolkit. Every clip is a per-joint linear combination of these envelopes; the windowed forms guarantee neutral start/end poses.

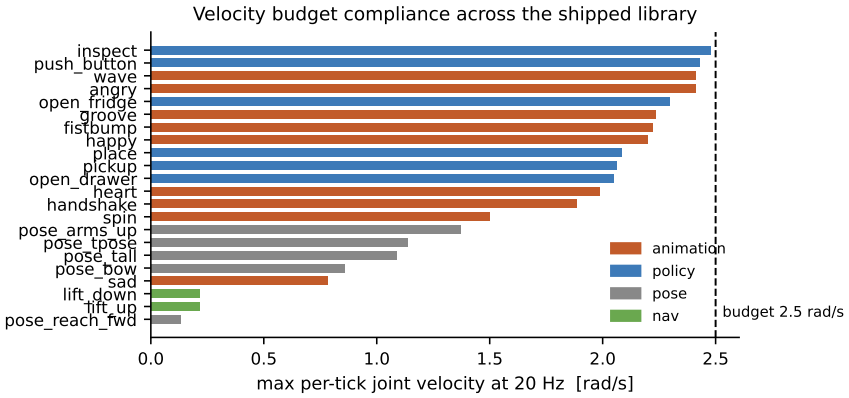


Figure 2: Maximum per-tick joint velocity of every executable action at 20 Hz sampling (grips scaled by 1.9 rad/unit). The whole library sits under the 2.5 rad/s budget; the bound is CI-enforced.

Backend merge. At boot the editor fetches `GET /api/actions` and merges: unknown ids appear as coming-soon slots (a backend `status:"available"` cannot make an action executable that has no local implementation — execution capability is local by construction); known ids may adopt display metadata only. With the endpoint absent the local registry stands alone. The live backend currently advertises 13 policy slots with plausible `location` labels (kitchen, laundry room, bedroom, . . .), rendered greyed with a *coming soon* tag.

Poses are keyframes too. Static poses expose both insertion modes in the menu: ♦ *key* adds the pose's target set as a v1 setpoint keyframe at the playhead (pose-to-pose interpolation on run), and □ *block* adds a transition-and-hold block (eased 1.6 s to 2.2 s transition, held for the block's trimmed length).

4 The NLE block model

A block is the tuple $b = (\text{clip}, t_0, \text{in}, \text{out}, \text{params}, \text{lane}, \text{linkId})$: it occupies timeline span $[t_0,\ t_0 + (\text{out} - \text{in})]$ and plays the clip-local slice $[\text{in}, \text{out}]$. Sampling at timeline time t evaluates the clip at

Channel	Range [rad]	Measured meaning
neutral / home	all zeros	both arms straight *forward*, palms down
{l,r}_arm_1	$\mp[-3.49, 1.40]$	shoulder pitch; left − / right + raises
{l,r}_arm_2	$\pm[-0.17, 3.32]$	abduction (T-pose at ± 1.45)
{l,r}_arm_3	$[\mp 1.57, \pm 1.57]$	humeral roll — rotates the elbow fold plane
{l,r}_arm_4	$\pm[-0.77, 1.67]$	elbow; left + / right − folds down/in
{l,r}_arm_5,7	$[-1.57, 1.57]$	forearm / wrist rolls
{l,r}_arm_6	$[-0.78, 0.78]$	wrist pitch (+ fingers down)
torso	$[-1.05, 0.26]$	− = forward lean (bow/slump), + = chest open
lift_lower+upper	$[0.155, 0.3175]$ each	*summed* into the one Lift actuator
{side}_grip	$[0, 1]$	whole-hand curl via the bridge's grasp map
neck (yaw/pitch/roll)	± 1.571, ± 0.785, ± 0.785	drivable (see caption)
navJoystick.ly	$[-1, 1]$	$v_x = 0.80$ ly m/s (fwd/rev symmetric)
navJoystick.lx	$[-1, 1]$	strafe, − = right, ≈ 0.87 m/s per unit
navJoystick.rx	$[-1, 1]$	yaw, $\omega = 1.50$ rx rad/s (− = CW)

Table 1: Empirical joint and drive conventions (probe sessions on the live sim). *Revised 2026-09.* The neck IS drivable from joint_command — it is only overridden while a live head block is streaming, which choreography is not — and the library now uses it rather than implying head intent through torso lean. The lift's lower bound is *not* its declared 0: the model's column self-penetrates the chassis below 0.155 m per stage, so only the upper half of the stroke is usable in simulation (controller/docs/rest_pose.tex, E4). An early reverse-drive estimate of 0.32 m/s per unit turned out to be the kitchen island 0.2 m behind the home spawn acting as a wall; open-floor re-measurement restored symmetry (§8.1).

local time

$$\tau \;=\; \mathrm{clamp}_{[0,\,d]}\big(\mathrm{in} + (t - t_0)\big), \tag{5}$$

where the clamp implements hold semantics past d.

Blade (B). Splitting b at playhead t_c produces $b_L = (\ldots, t_0, \mathrm{in}, \mathrm{in} + t_c - t_0)$ and $b_R = (\ldots, t_c, \mathrm{in} + t_c - t_0, \mathrm{out})$ — the two halves partition the clip range exactly, and each plays only its slice (unit-tested). Slivers under 0.1 s are refused. A cut block keeps the parent's clip key, so location metadata and module attribution compose automatically.

Trim. Dragging an edge moves in/out, clamped to $[0, d]$ for ordinary clips; hold clips may extend out arbitrarily (the pose holds). The left edge shifts t_0 equally so content stays anchored in time.

Move / snap. Drags resolve through a deterministic snapper: the candidate set is the 0.1 s grid plus t=0, the playhead, and every other block's edges; the nearest candidate within 0.12 s wins, grid otherwise. Identical inputs always produce identical placements (the Data Curation studio's "deterministic snap grid" feel).

Duplicate / delete / select. Alt-drag or the toolbar copies a block (params included); Delete/backspace removes the selected block or keyframe; selection is exclusive across both kinds.

Linked lanes. A lane:"both" clip (dances, drawer/fridge policies) instantiates two blocks sharing a linkId — the manipulation twin samples only joints, the navigation twin only nav; each is bladed/trimmed/moved independently.

Family	Actions (duration s)	Lane	Notes
Emotes	wave 5, handshake 6, fistbump 4.5, heart 8, happy 6, sad 7.5, angry 5	manip	validated live, ≥ 2 iterations each
Dances	groove 9, spin 8	both	base sway / full pirouette
Task policies	pickup 8^h, place 8, open_drawer 7, open_fridge 8, push_button 6, inspect 7	manip/ both	scripted loco-manip; drawer/fridge move the base
Nav primitives	fwd/back (m), strafe L/R (m), turn L/R (deg), hold (s)^h, goto (x,y)^h, lift up/down^h	nav	parametric, plant-calibrated; goto is REST closed-loop
Poses	home, tall, bow, arms-up, T-pose, reach	manip	hold blocks *or* keyframes
Coming soon	boxing, laundry, fold clothes, coffee, dishes, breakfast, charge, plants, ...	—	greyed, non-executable, carry `location` labels

Table 2: The shipped registry (^{h} = `hold`: sampling clamps past `dur`, so the block's right edge may be trimmed outward and the final pose / zero-stick is held).

Overlap rule. Within a lane the *last-starting* active block wins (top-most in NLE terms); nav-lane blocks that carry joints (the lift primitives) contribute to the joint stream as well.

Everything persists to `localStorage` (`mabel.cc.choreo.v1` for blocks, the v1 key for keyframes) and round-trips through JSON verbatim.

4.1 Expandable manipulation lane

The MANIPULATION header carries a disclosure toggle that expands six read-only module sub-tracks (left/right arm, left/right hand, torso·neck, lift). Module involvement is *derived*: each block's clip is sampled at nine points and a joint bucketed by id prefix marks its module if $|q_j| > 0.02$ anywhere — so the sub-tracks always tell the truth about what a block actually drives, including keyframes (rendered as slim segments and ticks; editing stays on the main lane).

5 The executor

Run streams the composition at 20 Hz (50 ms timer), gated by e-stop and Stiff mode exactly like v1:

- **Manipulation:** the commanded pose is $q(t) = q_{\text{keyframes}}(t) \oplus q_{\text{blocks}}(t)$ — keyframe interpolation underneath, block joints overriding per-joint. Identical consecutive poses are *not* re-sent (held poses go quiet on the wire).
- **Navigation:** while any nav-capable block exists the executor sends a `teleop_frame` *every tick*: the active block's `nav`(τ), zeros otherwise, and six zero-frames on stop — the wire contract is *never leave a non-zero last frame*. The always-on zero stream doubles as base pinning (§8.3).
- **Move-to-goal blocks** (`kind:"goal"`): block entry fires `POST /api/nav/goal {x,y}` to the native navigation service (macnav, the companion document), `GET /api/nav/status` is polled at ∼2 Hz for *reached/failed* toasts, and block exit or Stop fires `POST /api/nav/cancel`. A 404 degrades to a toast ("nav goal API not available"); the choreography plays on.

5.1 Physics preview: rehearse the plan in an isolated world

▶ Preview does NOT drive the robot: it rehearses the plan under *true MuJoCo physics* in a disposable twin. `POST /api/preview/start` spawns an ephemeral `simulation_bridge` instance (`--instance preview`: its own lock, state memmap and RPC port, so the one-physics-owner rule holds per world) *seeded from the live robot's current* `qpos`; a same-origin `/ws/preview` socket accepts the executor's exact wire envelopes (`joint_command`, `teleop_frame`, `reset`) and maps them onto the preview engine, and the browser MuJoCo viewer takes `?instance=preview` to render that world in a floating PHYSICS PREVIEW window. The timeline runs its ordinary 20 Hz loop against this sink — the only differences are that scalar {`side`}`_grip` values are expanded into explicit finger-joint targets client-side (the adapter maps canonical ids only) and that REST move-to-goal calls are *suppressed* (they would drive the LIVE navigator). A status chip reports *spinning up / running block n/m / done*; on completion zeros are streamed, the settled outcome stays visible for inspection, then the world resets for the next rehearsal (a 600 s idle timeout reclaims the process). Verified end-to-end: a wave + forward plan rehearsed under physics while the live robot's odometry moved $\leq$3 mm (operator noise), i.e. provably untouched. Scrubbing and *Space* keep the instant kinematic viewer preview (§10); ⇒ Execute alone commands the robot.

6 Location-aware gap validation

Actions may declare a `location {x, y, label}` — the spot they must run at. The validator walks all blocks in start order carrying the robot's *implied* location ℓ (initialized from live odometry): goal blocks *set* ℓ from their params; a located block *requires* $\|\ell - \ell_{\text{need}}\| < 0.35$ m (label equality when the backend supplies label-only locations). Each violation yields a gap $(t, \text{blockId}, \ell \to \ell_{\text{need}})$, rendered as a pulsing $\triangle$ marker on the navigation lane with two affordances: click $\to$ a pre-filled "insert: move to *kitchen*" confirmation, and *Auto-fix* (toolbar toggle, default on) which inserts the goto block itself, placed to *end* at the requiring block and sized by the distance estimate $\Delta t = \max(2,\ \|\Delta\ell\|/(0.80 \cdot 0.5) + 2)$ s. Validation re-runs on every edit; label-only locations (no map coordinates) are flagged but honestly not auto-fixable. Because all currently *available* actions are location-agnostic, gaps arise today only from future backend-declared located skills — the mechanism is fully unit-tested against synthetic located actions.

7 The validation loop (quality gate)

Every shipped clip passed a mandatory live-simulator loop before shipping:

1. **Sample** the clip at 20 Hz from the real `motions.js` (node), dump JSON;
2. **Replay** over the live wire (`ws://:9090/teleop`), with a full REST reset before and after, an eased lead-in, nav-zero base pinning, and wall-clock pacing;
3. **Capture** 3–8 stills from the server-rendered `/ws/simview` at expressive keypoints (freshness-checked frames) and *look at them*;
4. **Assert** from `robot_state` that the final arm pose is within 0.05 rad of target (sag-compensated);
5. **Refine and repeat** — at least two iterations per clip.

An offline *pose lab* (in-process MuJoCo, kinematics only, never touching the live state files) renders candidate keyposes and full trajectory contact sheets in seconds; final judgments always

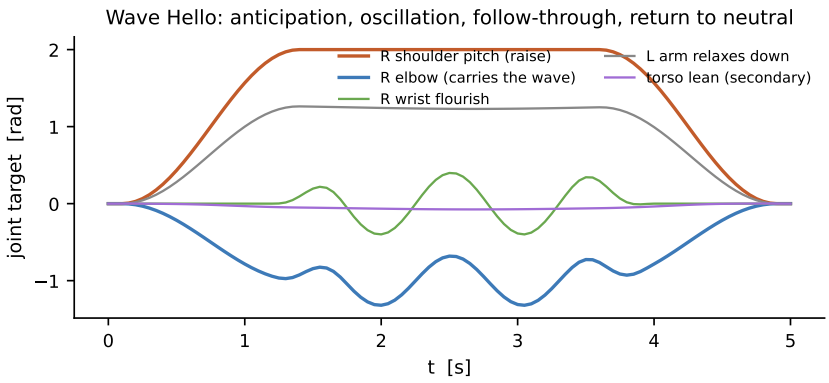


Figure 3: Anatomy of the shipped *Wave Hello*: an eased raise envelope, an elbow-carried oscillation with a wrist flourish, the off-arm relaxing down (asymmetry), a subtle torso lean, and an exact return to neutral.

came from the live robot. Iterations that mattered: the wave took four (shoulder geometry via candidate sheets, palm orientation via the humeral-roll trick of rotating the elbow's fold plane, base pinning); the heart took four (two candidate rounds converging on a face-level-then-overhead two-stage shape); sad took three (a self-hooking arm cleared by 0.22 rad of abduction); angry, inspect, handshake and fistbump three–four each; all remaining clips two. Fig. 3 shows the shipped wave's anatomy.

8 Experiments

All numbers from the 2026-07-02 live sessions (M-series Mac, `mabel_home` scene, 20 Hz command streams, odometry ground truth, full REST reset per trial); raw values in `experiments/results/choreo_measurements.json`, replays are wire playback of the sampled trajectories. Figures regenerate via `experiments/exp_choreography.py`.

8.1 Nav-primitive calibration

Fig. 4 and Table 3 summarize open-loop accuracy. The plant gains were measured first ($v = 0.80$ ly m/s, $\omega = 1.50$ rx rad/s from bridge drive telemetry), then the primitives' trapezoidal profiles were solved for duration analytically (the eased shoulders of a `win` envelope lose exactly r seconds of plateau area, which `durFor` adds back).

8.2 Tracking and return-to-neutral

Steady-state joint tracking under position hold is within 0.15 rad at gravity-loaded extremes (e.g. shoulder pitch commanded −1.20, read −1.05) and within 0.05 rad elsewhere; transient lag during a 2 rad raise peaks near 0.6 rad and converges in ∼1 s. Every animation clip ends with all 14 arm joints within 0.05 rad of neutral (the automated gate). Held poses sag consistently with gravity: T-pose abduction 0.104 rad, bow shoulders 0.135 rad, arms-up 0.088 rad — physics, not tracking failure, and left in the data unfudged.

Primitive	Commanded	Measured	Error	Note
forward	0.5 m	0.491 m	−1.8%	
back	0.5 m	0.529 m	+5.8%	open floor
back (from home)	0.5 m	0.195 m	—	island 0.2 m behind: a wall, not a plant asymmetry
strafe left / right	0.4 m	0.438 m	+9.5%	strafe gain ≈0.87 m/s/unit
turn left / right	90°	88.5° / 88.8°	−1.6%	
turn	180°	159.7°	−11%	open-loop shortfall grows with angle (§8.4)
spin (dance)	360°	357.6°	−0.7%	windowed $\int \omega\, dt$ design
stop & hold	0 m	0.000 m	—	active zero stream

Table 3: Open-loop nav primitives against odometry. The "back from home" row is kept as a finding: an apparent 2.5× reverse asymmetry disappeared once the trial start cleared the kitchen island — environment contact, not dynamics. Precise long rotations should use the closed-loop `move-to-goal` (macnav) instead of open-loop turns.

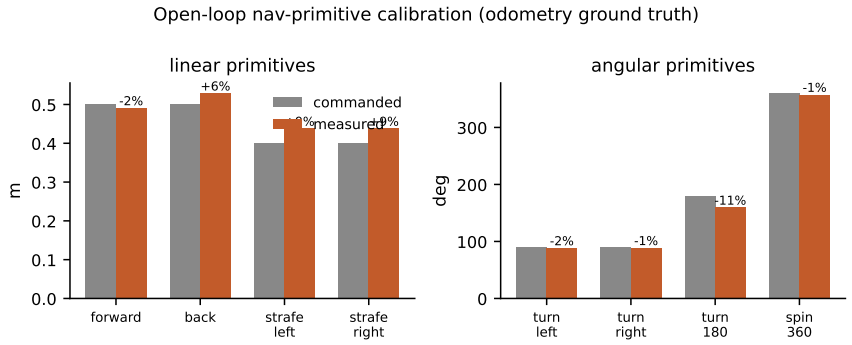


Figure 4: Commanded vs. measured displacement for the parametric nav primitives (odometry ground truth, one representative trial each).

8.3 Base pinning during manipulation

With no nav stream, arm reaction forces walk the free-castering swerve base: the wave alone displaced the base 0.221 m. Streaming `navJoystick` zeros alongside the joint stream (now the executor default) pins the same clip to 0.002 m. This is the single highest value-per-line finding of the validation campaign.

8.4 End-to-end choreography

The gate composition — wave at 0.5 s, forward 0.8 m at 6 s, pick-up at 9.5 s, 180° turn at 18 s, happy bounce at 23.5 s; 29.5 s total, built on the real `ChoreoTrack` and streamed by the real sampling path — executed cleanly: translation 0.772 m (−3.5%), final arms neutral within 0.05 rad, the held pick-up carry persisting through the turn as intended. The in-context turn reached 107° vs. 159.7° isolated: large open-loop turns lose authority in tight quarters and while carrying — quantified here as a design bound, with closed-loop goto as the prescribed alternative. Suppressing re-sends of identical held poses (the executor rule above) was tested as a candidate cause and cleared: the shortfall is physical.

8.5 Machine-checkable gate

`node --test simulation_control_center/tests/js/*.test.mjs`: 95 pass, 0 fail — schema completeness; joints within calibrated ranges over full sampling (hand ranges from the ORCA calibration); the velocity budget; neutral start/end for animations; steady, clamped holds; nav sticks bounded with zero endpoints; blade/trim/snap/duplicate semantics; linked-lane instantiation; JSON round-trips; gap detection, auto-fix sizing and label-only refusal; and backend-merge safety (a backend cannot promote a slot to executable). The v1 sequence tests pass untouched.

9 Implementation and usage

```
# run the studio (timeline at the bottom of the page)
./simulation_control_center/run.sh                        # -> http://mac:8097
# tests (pure node, no robot needed)
node --test simulation_control_center/tests/js/*.test.mjs
# regenerate every figure in this document
python3 simulation_control_center/experiments/exp_choreography.py
```

The toolbar is grouped add · edit · view · transport: *+ Nav / + Policy / + Animation / + Pose / + Text* open per-family menus at the playhead (parametric actions prompt for values; the +Pose menu also carries the live-capture keyframe tools; +Text adds a language annotation to the LANGUAGE lane, where action blocks already render their derived labels). Drag blocks (snap), drag edges (trim), *B* blades at the playhead, alt-drag duplicates, *Delete* removes, *Space* plays the 3D *preview*; only ⇒ Execute commands the robot. The Auto-fix toggle governs gap insertion. Blocks, keyframes, and annotations persist per browser.

10 Limitations

- **The neck does not participate.** `joint_command` has no neck route (head-tracking owns it); "look" intent is faked with torso and arm motion. A neck route in the facade would materially improve the emotes.
- **Open-loop turns degrade with angle and context** (-11% at $180°$ isolated, -40% in the E2E's tight quarters). The composer's answer is `move-to-goal`; a yaw-odometry-closed turn primitive would be the next step.
- **Clips are environment-blind.** Self-collision avoidance is the bridge's; furniture is not consulted. Big gestures need $\sim$0.7 m of clearance; the home spawn's island taught us this the hard way (Table 3).
- **Grip is one scalar per hand** in the emote set (individual fingers are used only where it matters, e.g. the button-press point); expressive finger choreography is future work.
- **The 3D preview is kinematic.** Scrub / ▶ Preview plays the choreography on the main viewer rig (joints + the nav-integrated base path) without commanding the robot; only ⇒ Execute streams to the wire, fading the viewer back to the live state. The preview integrates the *calibrated* drive gains, not contact physics, so environment collisions and turn shortfall (§8.4) do not appear in it.
- **Location auto-fix needs coordinates.** Backend label-only locations flag gaps but cannot be auto-bridged until the backend ships map coordinates for its labels.

Companion documents

`simulation_control_center.pdf` (the studio platform), `macnav.pdf` (the native navigation/SLAM service behind `move-to-goal`), `retargeting.pdf` (webcam body teleop), `controller/docs/mabel_controller.pdf` (the teleop controller whose `joint_command` route this composer drives).

References

[1] J. Lasseter, "Principles of traditional animation applied to 3D computer animation," in *Proc. SIGGRAPH*, 1987.

[2] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ IROS*, 2012.

[3] C. Breazeal, "Toward sociable robots," *Robotics and Autonomous Systems*, 42(3–4):167–175, 2003.

[4] T. Ribeiro and A. Paiva, "The illusion of robotic life: principles and practices of animation for robots," in *ACM/IEEE HRI*, 2012.

[5] M. Suguitan and G. Hoffman, "Blossom: A handcrafted open-source robot," *ACM Transactions on Human-Robot Interaction*, 8(1), 2019.